



INDIVIDUAL ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT074-3-2-CCP

CONCURRENT PROGRAMMING

HAND OUT DATE: 16 March 2026

HAND IN DATE: 24 May 2026

WEIGHTAGE: 45%

Name	TP Number	Intake Code
Mohib Bin Aamir	TP085584	APD2F2509CS(DA

Table of Contents

1.0 Introduction.....	
2.0 Background of the Airport Simulation.....	
3.0 Assumptions.....	
4.0 Basic Requirements Met.....	
4.1 One Runway for Landing and Take-Off.....	
4.2 Maximum Three Aircraft on Airport Grounds.....	
4.3 Complete Aircraft Process from Landing to Take-Off.....	
4.4 No Waiting Area on the Ground.....	
4.5 Final Sanity Check and Statistics Report.....	
5.0 Additional Requirements Met.....	
5.1 One Refueling Truck.....	
5.2 Congested Scenario with Waiting Aircraft.....	
5.3 Emergency Landing Priority.....	
5.4 Six Aircraft with Random Arrival Time.....	
5.5 Concurrent Ground Servicing Operations.....	
6.0 Safety Aspects of the Multi-Threaded System.....	
7.0 Justification of Coding Techniques Implemented.....	
8.0 Depth of Discussion of Concurrency Concepts.....	
9.0 Requirements Not Met.....	
10.0 Output of the Program.....	
11.0 Conclusion.....	
12.0 References.....	

1.0 Introduction

The Asia Pacific Airport Simulation designed for the Concurrent Programming assignment is explained in this report. This system's aim is for replicating the safe arrival, landing, usage of airport resources, ground service, as well as takeoff of airplane. The significant concurrent programming concepts are threads, synchronization, mutual exclusion, wait() and notifyAll() have been used in this Java program.

All aircraft can be seen as a separate thread in this simulation. There is one runway, three gates, and one refueling truck at the airport. Shared resources should be carefully handled to avoid crashes, race situations, and incorrect output because multiple planes are operating at the same time. An emergency landing scenario where an aircraft having a fuel shortage is given priority is additionally included in the simulation.

2.0 Background of the Airport Simulation

A small airport setting where several planes compete regarding limited resources is used as the basis of the airport simulation. Only a single plane can land or take off at a time as the airport has only one runway. The airport can fit a maximum of three planes on the ground, which includes the runway and has three gates available regarding plane servicing.

When a plane gets the granted permission to land, it goes to a designated gate, docks, allows passengers to depart, gets cleaned, refills food and supplies, and eventually asks to take off. These processes match the actual airport operations in a simplified way. The system is meant to illustrate how java thread synchronization could be used to safely handle concurrent tasks.

3.0 Assumptions

- The airport has single runway.
- The airport has three gates.
- There is one refueling truck only.
- The simulation has used six aircraft.
- Each aircraft carries up to 50 passengers.
- Passenger numbers are randomly generated.

- Passengers are always ready in the terminal.
- Aircraft can land only if a runway and gate are free.
- There is no ground waiting area.
- One aircraft is considered an emergency plane.
- The simulation runs below 60 seconds.

4.0 Basic Requirements Met

4.1 One Runway for Landing and Take-Off

```
while (!runwayAvailable) {
    try {
        System.out.println("ATC: " + aircraftName
            + " waiting for runway to become free for take-off.");
        wait();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

runwayAvailable = false;
System.out.println("ATC: Take-off permission granted for " + aircraftName + ".");
```

Figure 4.1: Runway Synchronization for Aircraft Take-Off

One shared runway regarding every plane is effectively carried out via the program. The runwayAvailable variable inside the Airport class is implemented to control this. The system checks whether the runway is available before a plane can land or take off. The plane needs to wait if the runway is busy. This prevents two aircrafts from using the runway at the same time.

4.2 Maximum Three Aircraft on Airport Grounds

```
if (aircraftOnGround >= maxAircraftOnGround) {
    return false;
}
```

Figure 4.2: Ground Capacity Control for Maximum Three Aircraft

Three aircraft are given permission on the airport ground according to the system. Aircraft at the gates and on the runway fall within this category. While landing permission is given, the

variable `aircraftOnGround` rises, and when takeoff is completed, it reduces. This obeys safety regulations and prevents airport traffic.

4.3 Complete Aircraft Process from Landing to Take-Off

```
Gate assignedGate = coordinator.requestLanding(aircraftName, emergency);

land();
coordinator.releaseRunwayAfterLanding(aircraftName);

coastToGate(assignedGate);
dockAtGate(assignedGate);

performGroundServices();

undockFromGate(assignedGate);
coastToRunway();

coordinator.requestTakeOff(aircraftName, assignedGate);
takeOff();
coordinator.completeTakeOff(aircraftName, assignedGate);
```

Figure 4.3: Complete Aircraft Operation Sequence

All planes obey the whole process required by the assignment. Upon requesting landing, the plane goes to a gate, docks, complete ground services, undocks, goes back to the runway, and takes off. The Aircraft class's `run()` method includes this sequence.

4.4 No Waiting Area on the Ground

```
if (getFreeGate() == null) {
    return false;
}
```

Figure 4.4: Gate Availability Check Before Landing

Based on the simulation, planes are not permitted to wait for a gate on the airport ground. The program checks whether a gate is open before given permission for landing. The plane waits in the air before the gate opens, if the gate is not open.

4.5 Final Sanity Check and Statistics Report

```

public synchronized void performSanityCheck() {
    System.out.println("\n----- Sanity Check -----");

    boolean allGatesEmpty = true;

    for (Gate gate : gates) {
        if (gate.isOccupied()) {
            allGatesEmpty = false;
            System.out.println("Gate-" + gate.getGateNumber() + " is still occupied.");
        } else {
            System.out.println("Gate-" + gate.getGateNumber() + " is empty.");
        }
    }

    System.out.println("All gates empty: " + allGatesEmpty);
    System.out.println("-----");
}

```

Figure 4.5: Final Sanity Check

```

public synchronized void printStatistics() {
    System.out.println("\n----- Statistics Report -----");

    if (waitingTimes.isEmpty()) {
        System.out.println("No waiting time data available.");
    } else {
        long min = waitingTimes.get(0);
        long max = waitingTimes.get(0);
        long total = 0;

        for (long time : waitingTimes) {
            if (time < min) {
                min = time;
            }

            if (time > max) {
                max = time;
            }

            total += time;
        }

        double average = (double) total / waitingTimes.size();

        System.out.println("Minimum waiting time: " + min + " ms");
        System.out.println("Average waiting time: " + String.format("%.2f", average) + " ms");
        System.out.println("Maximum waiting time: " + max + " ms");
    }

    System.out.println("Number of aircraft served: " + aircraftServed);
    System.out.println("Total passengers boarded: " + passengersBoarded);
    System.out.println("-----");
}

```

Figure 4.6: Final Statistics Report

The airport runs a sanity check to ensure all gates are free at the end of the simulation. The statistics report additionally illustrates the overall number of passengers boarded, the number of planes served, and the minimum, average, and maximum waiting periods.

5.0 Additional Requirements Met

5.1 One Refueling Truck

```
public synchronized void refuel(String aircraftName) throws InterruptedException {
    System.out.println(aircraftName + ": Waiting for refuel truck.");
    System.out.println("Refuel Truck: Refueling " + aircraftName + ".");
    Thread.sleep(1500);
    System.out.println("Refuel Truck: Refueling completed for " + aircraftName + ".");
}
```

Figure 5.1: Synchronized Refueling Truck Operation

There is just a single refueling truck in the simulation. Only a single plane at a time as the `refuel()` method in the `RefuelTruck` class is classified as synchronized. Unless the truck is occupied, another plane should wait.

5.2 Congested Scenario with Waiting Aircraft

```
while (!canLand(emergency)) {
    try {
        if (aircraftOnGround >= maxAircraftOnGround) {
            System.out.println("ATC: Landing permission denied for "
                + aircraftName + ", airport ground is full.");
        } else if (!runwayAvailable) {
            System.out.println("ATC: " + aircraftName
                + " waiting because runway is occupied.");
        } else if (getFreeGate() == null) {
            System.out.println("ATC: " + aircraftName
                + " waiting because no gate is available.");
        } else if (!emergency && emergencyWaiting) {
            System.out.println("ATC: " + aircraftName
                + " waiting because emergency aircraft has priority.");
        }

        wait();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

Figure 5.2: Aircraft Waiting During Airport Congestion

When aircraft wait because of the shortage of runway, or airport ground capacity, the program displays congestion. The output shows ATC messages, like aircraft waiting due to the airport ground is packed or the runway is occupied.

5.3 Emergency Landing Priority

```
if (!emergency && emergencyWaiting) {  
    return false;  
}
```

Figure 5.3: Emergency Aircraft Priority Handling

The simulation has an emergency aircraft to simulate a shortage of fuel. The variable `emergencyWaiting` has been set to true when the emergency aircraft issues a landing request. Whereas the emergency aircraft are given priority, the usual aircraft needs to wait.

5.4 Six Aircraft with Random Arrival Time

```
Aircraft[] aircraftList = new Aircraft[6];
Random random = new Random();

for (int i = 0; i < aircraftList.length; i++) {
    String aircraftName = "Aircraft-" + (i + 1);

    // Aircraft-5 is used as the emergency aircraft.
    boolean emergency = (i == 4);

    int passengersToDisembark = random.nextInt(50) + 1;
    int passengersToBoard = random.nextInt(50) + 1;

    aircraftList[i] = new Aircraft(
        aircraftName,
        emergency,
        passengersToDisembark,
        passengersToBoard,
        coordinator,
        refuelTruck,
        statistics
    );
}

for (Aircraft aircraft : aircraftList) {
    aircraft.start();

    try {
        // New aircraft arrives every 0, 1, or 2 seconds.
        Thread.sleep(random.nextInt(3) * 1000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

Figure 5.4: Six Aircraft Threads with Random Arrival Time

Six aircraft threads are generated via the program. There is a random delay of 0,1, or 2 seconds while each aircraft comes. Aircrafts do not come at the same time, which adds reality to the scenario.

5.5 Concurrent Ground Servicing Operations

```
passengerThread.start();
cleaningThread.start();
refuelThread.start();

passengerThread.join();
cleaningThread.join();
refuelThread.join();
```

Figure 5.5: Concurrent Ground Services at Airport Gates

Separate threads are utilized at the gate regarding passenger service, cleaning, and supply restocking as well as fueling. This makes it possible to perform various activities at once while the aircraft is docked.

6.0 Safety Aspects of the Multi-Threaded System

Several aircraft threads trying to use shared airport resources at the same time are the key safety risks in this scenario. The runway, gates, aircraft ground count, emergency priority status, refueling truck, and statistics are a few of these usual resources. Race conditions can increase if these resources are not properly handled.

The application uses synchronization methods in Statistics, RefuelTruck, and Airport classes to prevent this. Only a single thread could update important shared data at a time because of synchronization. For example, synchronized methods manage ground capacity, runway status, permission for landing, and gate assignment. This avoids situations in which two aircraft use the same runway or are given the same gate.

Furthermore, safety is improved via using `wait()` and `notifyAll()`. When the runway is blocked, the airport is full or there is no gate available, aircraft wait. Waiting threads are notified and given permission to recheck the status when a runway or gate is available. Due to this the aircraft only continue when the required condition is fulfilled, this makes the simulation secure.

7.0 Justification of Coding Techniques Implemented

Due to the assignment, it calls for a concurrent simulation; the system uses java threads. The Thread class has been expanded to create a unique thread for each aircraft. This is correct because each aircraft accomplishes its unique set of activities on its own.

To protect shared resources, the synchronized keyword is used. This is significant as several aircraft cannot use the runway, gates, and refueling truck simultaneously. When an aircraft

cannot move forward safely, the wait() method is used; when the condition of the airport changes, notifyAll() is used.

Furthermore, instead of using constrained automatic concurrency libraries, the program uses basic arrays and variables. This keeps the concurrency logic's manual implementation and clarity. During aircraft servicing, concurrent activity is illustrated via the use of separate threads regarding passenger service, cleaning, supply restocking and refueling.

8.0 Depth of Discussion of Concurrency Concepts

This program uses thread-based execution as its key concurrency concept. Aircraft activities could overlap as each aircraft operates autonomously. To secure shared data and end unauthorized entry, synchronization is used.

The refueling truck and the runway are considered mutual exclusion. This indicates that only single aircraft at a time might take off, land, or refuel. As variables like aircraftOnGround must be updated consistently, atomic updates are additionally important. The result can be wrong if two threads change this value simultaneously with no synchronization.

Furthermore, the program uses wait() and notifyAll() to illustrate inter-thread communication. Aircraft threads are told when a resource is available and wait when the airport cannot receive them. This makes it possible for the simulation to operate securely with no inadequate monitoring status every time.

9.0 Requirements Not Met

All requirements are fulfilled.

10.0 Output of the Program

```
=====
      Asia Pacific Airport Simulation
=====

Aircraft-1: Requesting landing.
ATC: Landing permission granted for Aircraft-1.
ATC: Gate-1 assigned for Aircraft-1.
Aircraft-1: Landing.
Aircraft-1: Landed.
```

Figure 10.1: Aircraft Landing Request and ATC Gate Assignment

```
Aircraft-5: Fuel shortage! Emergency landing required.
ATC: Landing permission denied for Aircraft-6, airport ground is full.
Aircraft-5: Requesting landing.
ATC: Emergency detected for Aircraft-5 due to fuel shortage.
ATC: Landing permission denied for Aircraft-5, airport ground is full.
Aircraft-2 Passengers: 19 passengers embarking.
Aircraft-3 Passengers: 32 passengers disembarking.
Aircraft-3 Cleaning Crew: Cleaning aircraft.
Aircraft-2 Supply Crew: Refilling food and supplies.
Aircraft-1: Requesting take-off.
ATC: Take-off permission granted for Aircraft-1.
Aircraft-1: Taking off.
Refuel Truck: Refueling completed for Aircraft-2.
Aircraft-3: Waiting for refuel truck.
Refuel Truck: Refueling Aircraft-3.
Aircraft-2 Passengers: Passenger service completed.
Aircraft-3 Passengers: 31 passengers embarking.
Aircraft-2 Cleaning/Supply Crew: Service completed.
Aircraft-2: All ground services completed.
Aircraft-2: Undocking from Gate-2.
Aircraft-3 Supply Crew: Refilling food and supplies.
Aircraft-1: Take-off completed.
ATC: Aircraft-1 has departed.
ATC: Gate-1 is now empty.
ATC: Runway is now free.
ATC: Aircraft-4 waiting because emergency aircraft has priority.
ATC: Aircraft-6 waiting because emergency aircraft has priority.
ATC: Landing permission granted for Aircraft-5.
ATC: Gate-1 assigned for Aircraft-5.
Aircraft-5: Landing.
```

Figure 10.2: Emergency Aircraft Landing Priority Output

```
Aircraft-2 Passengers: 19 passengers embarking.  
Aircraft-3 Passengers: 32 passengers disembarking.  
Aircraft-3 Cleaning Crew: Cleaning aircraft.  
Aircraft-2 Supply Crew: Refilling food and supplies.  
Aircraft-1: Requesting take-off.
```

Figure 10.3: Concurrent Ground Services Output

```
=====
                        Simulation Finished
=====

----- Sanity Check -----
Gate-1 is empty.
Gate-2 is empty.
Gate-3 is empty.
All gates empty: true
-----

----- Statistics Report -----
Minimum waiting time: 0 ms
Average waiting time: 1991.17 ms
Maximum waiting time: 5659 ms
Number of aircraft served: 6
Total passengers boarded: 150
-----
```

Figure 10.4: Final Statistics Report and Airport Sanity Check

Aircraft requesting landing, ATC giving permission, gate assigning, aircraft docking, passenger movement, cleaning, supply restocking, refueling, takeoff, and final departure are all shown in the program's output. Also, the output shows the emergency aircraft's detection and priority handling. The sanity check confirms that Gates 1, 2, and 3 are empty at the end of the simulation. The final statistics report shows the overall number of aircraft served, the number of passengers boarded, and the minimum, average, and maximum waiting times.

11.0 Conclusion

Thus, the Asia Pacific Airport Simulation efficiently demonstrates how to use Java for concurrent programming. The system uses synchronization to secure shared resources, which includes the runway, gates, and refueling truck while simulating original airport operations using multiple aircraft threads. Waiting, notification, emergency priority, random aircraft arrival, concurrent maintenance, final statistics, and sanity tests are all implemented via the application. With everything considered, the simulation meets the assignment's basic and additional

requirements. It illustrates how concurrency could improve realism in a multithreaded system while maintaining security.

12.0 References

GeeksforGeeks. (2026). *Synchronization in Java*. GeeksforGeeks.

<https://www.geeksforgeeks.org/java/synchronization-in-java/>

GeeksforGeeks. (2026). *Multithreading in java*. GeeksforGeeks.

<https://www.geeksforgeeks.org/java/multithreading-in-java/>

Thread synchronization in Java. (2026). Medium- Nakul Mitra.

<https://medium.com/towardsdev/thread-synchronization-in-java-73116ee58ac9>